

Solution Homework 3

Spring 2025

Exercise 1 (20 pt), Electron moving into an electric field

An electron is sent through a varying electrical field. Initially, the electron is moving in the x -direction with a velocity $v_x = 100$ m/s. The electron enters the field when it passes the origin. The field varies with time, causing an acceleration of the electron that varies in time

$$\mathbf{a}(t) = \left(-20\text{m/s}^2 - 10\text{m/s}^3 t \right) \mathbf{e}_y$$

- 1a (4pt) Find the velocity as a function of time for the electron.

Exercise 2 (10 pt), Drag force

We can observe that the models for linear and quadratic drag forces are given by:

$$f_{lin} = 3\pi\eta Dv \quad f_{quad} = \kappa\rho A v^2$$

where D is the "length scale" of the object (e.g., the diameter of the sphere), η is the viscosity of the fluid, ρ is the density of the fluid, A is the cross-sectional area of the object, and κ is a constant of order unity (larger for flat and blunt bodies; smaller for streamlined bodies).

- 2a (5pt) The Reynolds number is defined as $Re = \rho v D / \eta$. What is the physical meaning of this number? For a spherical object, show that the ratio of the quadratic drag force to the linear drag force is given by $f_{quad}/f_{lin} = Re/48$. Use this to explain the physical meaning of the Reynolds number. **Note: you may assume that $\kappa = 0.25$ for a sphere.**

Exercise 3 (10 pt), Falling object

We have shown that an object that is dropped from rest and experiences linear drag force will reach a terminal velocity and do so exponentially:

$$v_y(t) = v_{ter}(1 - e^{-t/\tau}).$$

- 3a (4pt) At first, the object will be moving slowly. Show that we can approximate this expression in that interval. We should find that $v_y = gt$ -- the standard result for free fall in vacuum. Demonstrate this.

Exercise 4 (10 pt), and now a theoretical exercise

Finding and exploring equations of motion is the central enterprise of classical mechanics. You will encounter (and derive) many equations you have not seen before, and you will need to explain them. Consider a hypothetical one-dimensional system where a mass m has a speed of v_0 and coasts along the x-axis. The surrounding medium produces a drag force that is modeled using:

$$F(v) = -cv^{3/2}.$$

When the force is a pure function of velocity, using the technique called separation of variables on Newton's 2nd Law, we can find the velocity as a function of time:

We can separate variables and write:

$$F(v) = ma = m \frac{dv}{dt}$$

Divide both sides by $F(v)$ and multiply both sides by dt :

$$dt = \frac{m}{F(v)} dv$$

And then, given starting (initial) conditions, we integrate both sides to find an expression for t as a function of v .

$$t = m \int_{v_0}^v \frac{dv'}{F(v')}$$

- 4a (4pt) For the given force above, write the equation of motion in a form: $\frac{mdv}{F(v)} = dt$. Integrate both sides to find an expression for the velocity v as a function of t ($v(t)$ not $t(v)$).

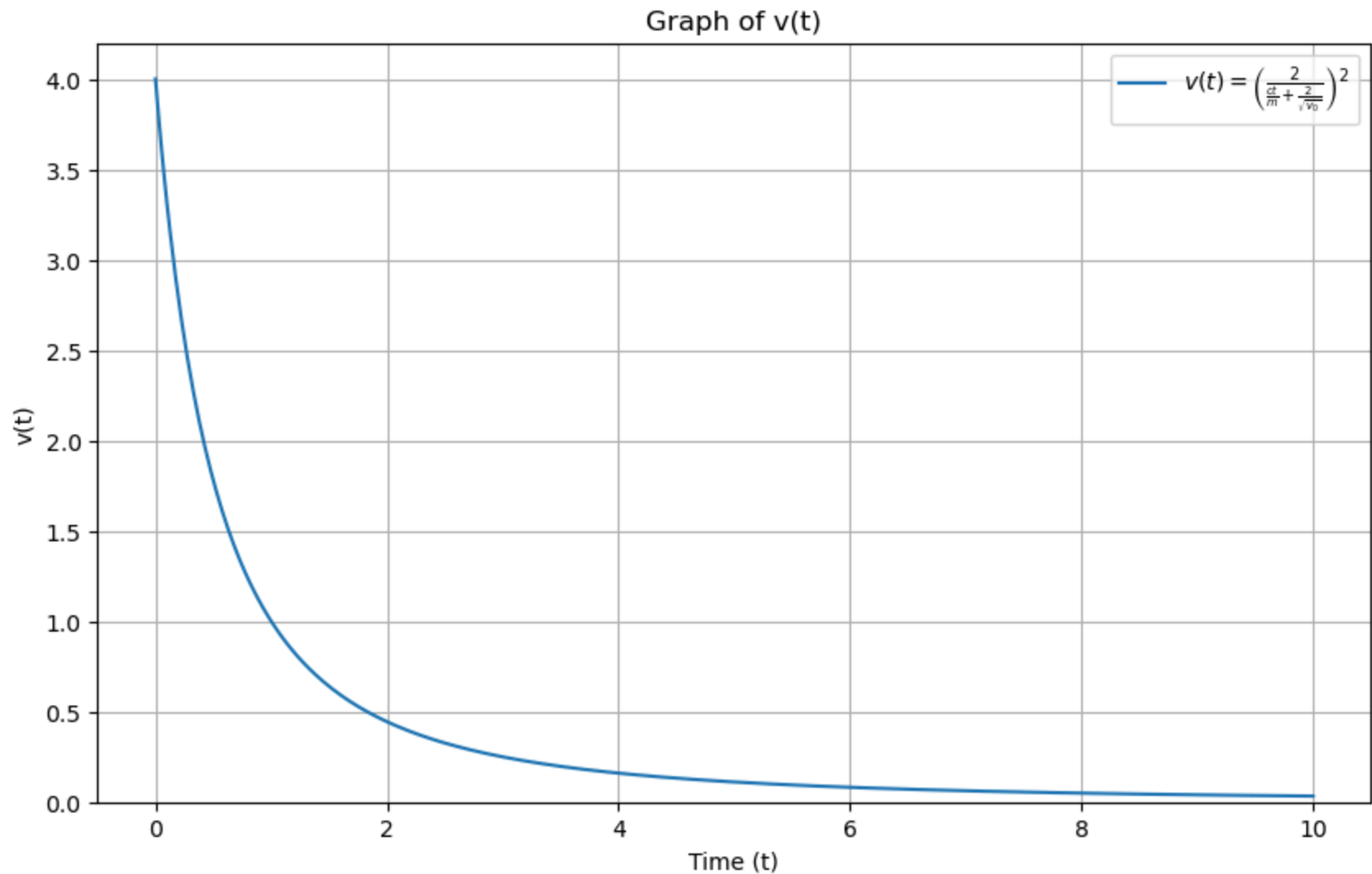
```
In [1]: import matplotlib.pyplot as plt
import numpy as np

# Constants
c = 1 # Just for illustration, the actual value depends on the specific problem
m = 1 # Same as above
v0 = 4 # A positive initial value for v0

# Time array
t = np.linspace(0, 10, 400)

# Function v(t)
v_t = (2 / (c * t / m + 2 / np.sqrt(v0)))**2

# Plot
plt.figure(figsize=(10, 6))
plt.plot(t, v_t, label=r"$v(t) = \left( \frac{2}{\frac{ct}{m} + \frac{2}{\sqrt{v_0}}} \right)^2$")
plt.xlabel('Time (t)')
plt.ylabel('v(t)')
plt.title('Graph of v(t)')
plt.ylim(bottom=0)
plt.grid(True)
plt.legend()
plt.show()
```



- 4d (2pt) Will the object ever come to rest?

Exercise 5 (10 pt), back to a falling ball and preparing for the numerical exercise

Useful material: Malthe-Sørenssen chapter 7.5 and Taylor chapter 2.4.

In this example we study the motion of an object subject to a constant force and a velocity dependent force. We will reuse the code we develop here in homework 4 for a position-dependent force.

Here we limit ourselves to a ball that is thrown from a height h above the ground with an initial velocity $\mathbf{v}_0$ at time $t = t_0$. We assume the air resistance is proportional to the square velocity. Together with the gravitational force these are the forces acting on our system.

$$\mathbf{F}_{grav} = -mg\mathbf{e}_y$$

and an air resistance given by a square law

$$\mathbf{F}_{drag} = -Dv\mathbf{v}.$$

The analytical expressions for velocity and position as functions of time will be used to compare with the numerical results in exercise 6.

- 5a (3pt) Identify the forces acting on the ball and set up a diagram with the forces acting on the ball. Find the equation of motion for the falling ball. **Do not limit to 1D yet!**

Exercise 6 (40pt), Numerical elements, solving exercise 5 numerically

This exercise should be handed in as a jupyter-notebook at D2L. Remember to write your name(s).

Last week we:

1. Gained more practice with plotting in Python
2. Became familiar with arrays and representing vectors with such objects

This week we will:

1. Learn and utilize Euler's Method to find the position and the velocity

2. Compare analytical and computational solutions
3. Add additional forces to our model

```
In [1]: %matplotlib inline

# let's start by importing useful packages we are familiar with
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
```

We will choose the following values

1. mass $m = 0.2$ kg
2. acceleration (gravity) $g = 9.81$ m/s².
3. initial position is the height $h = 2$ m
4. initial velocities $v_{x,0} = v_{y,0} = 10$ m/s

Can you find a reasonable value for the drag coefficient D ? You need also to define an initial time and the step size Δt . We can define the step size Δt as the difference between any two neighboring values in time (time steps) that we analyze within some range. It can be determined by dividing the interval we are analyzing, which in our case is time $t_{\text{final}} - t_0$, by the number of steps we are taking (N). This gives us a step size $\Delta t = \frac{t_{\text{final}} - t_0}{N}$.

With these preliminaries we are now ready to plot our results from exercise 5.

- 6a (10pt) Set up arrays for time, velocity, acceleration and positions for the results from exercise 5. Define an initial and final time. Choose the final time to be the time when the ball hits the ground for the first time. Make a plot of the position and velocity as functions of time. Here you could set the initial velocity in the y -direction to zero and use the result from exercise 5. Else you need to try different initial times using the result from exercise 5 as a starting guess. It is not critical if you don't reach the ground when the initial velocity in the y -direction is not zero.

We move now to the numerical solution of the differential equations as discussed in the [lecture notes](#) or Malthe-Sørensen chapter 7.5. Let us remind ourselves about Euler's Method.

Suppose we know $f(t)$ and its derivative $f'(t)$. To find $f(t + \Delta t)$ at the next step, $t + \Delta t$, we can consider the Taylor expansion:

$$f(t + \Delta t) = f(t) + \frac{(\Delta t)f'(t)}{1!} + \frac{(\Delta t)^2 f''(t)}{2!} + \dots$$

If we ignore the f'' term and higher derivatives, we obtain

$$f(t + \Delta t) \approx f(t) + (\Delta t)f'(t).$$

This approximation is the basis of Euler's method, and the Taylor expansion suggests that it will have errors of $O(\Delta t^2)$. Thus, one would expect it to work better, the smaller the step size h that you use. In our case the step size is Δt .

In setting up our code we need to

1. Define and obtain all initial values, constants, and time to be analyzed with step sizes as done above (you can use the same values)
 2. Calculate the velocity using $v_{i+1} = v_i + (\Delta t) * a_i$
 3. Calculate the position using $pos_{i+1} = r_i + (\Delta t) * v_i$
 4. Calculate the new acceleration a_{i+1} .
 5. Repeat steps 2-4 for all time steps within a loop.
- 6b (20 pt) Write a code which implements Euler's method and compute numerically and plot the position and velocity as functions of time for various values of Δt . Comment your results.
 - 6c (10pt) Compare your numerically obtained positions and velocities with the analytical results from exercise 5. In order to do this, you need to take out the motion in the x -direction. Comment again your results.

```
In [15]: # Exercise 6, hw3, brute force way with declaration of vx, vy, x and y
# Common imports
import numpy as np
import pandas as pd
```

```

from math import *
import matplotlib.pyplot as plt
import os

# Where to save the figures and data files
PROJECT_ROOT_DIR = "Results"
FIGURE_ID = "Results/FigureFiles"
DATA_ID = "DataFiles/"

if not os.path.exists(PROJECT_ROOT_DIR):
    os.mkdir(PROJECT_ROOT_DIR)

if not os.path.exists(FIGURE_ID):
    os.makedirs(FIGURE_ID)

if not os.path.exists(DATA_ID):
    os.makedirs(DATA_ID)

def image_path(fig_id):
    return os.path.join(FIGURE_ID, fig_id)

def data_path(dat_id):
    return os.path.join(DATA_ID, dat_id)

def save_fig(fig_id):
    plt.savefig(image_path(fig_id) + ".png", format='png')

# Output file
outfile = open(data_path("Eulerresults.dat"), 'w')

from pylab import plt, mpl
plt.style.use('ggplot')
mpl.rcParams['font.family'] = 'serif'

g = 9.80655 #m/s^2
# The mass and the drag constant D
D = 0.00245 #mass/length kg/m
m = 0.2 #kg, mass of falling object
DeltaT = 0.001
#set up arrays
tfinal = 1.4

```



```

# set up number of points for all variables
n = ceil(tfinal/DeltaT)
# define scaling constant vT used in analytical solution
vT = sqrt(m*g/D)
# set up arrays for t, a, v, and y and arrays for analytical results
#brute force setting up of arrays for x and y, vx, vy, ax and ay
t = np.zeros(n)
vy = np.zeros(n)
y = np.zeros(n)
vx = np.zeros(n)
x = np.zeros(n)
yanalytic = np.zeros(n)
# Initial conditions, note that these correspond to an object falling in the y-direction only.
vx[0] = 0.0 #m/s
vy[0] = 0.0 #m/s
y[0] = 10.0 #m
x[0] = 0.0 #m
yanalytic[0] = y[0]
# Start integrating using Euler's method
for i in range(n-1):
    # expression for acceleration, note the absolute value and division by mass
    ax = -D*vx[i]*sqrt(vx[i]**2+vy[i]**2)/m
    ay = -g - D*vy[i]*sqrt(vx[i]**2+vy[i]**2)/m
    # update velocity and position
    vx[i+1] = vx[i] + DeltaT*ax
    x[i+1] = x[i] + DeltaT*vx[i]
    vy[i+1] = vy[i] + DeltaT*ay
    y[i+1] = y[i] + DeltaT*vy[i]
    # update time to next time step and compute analytical answer
    t[i+1] = t[i] + DeltaT
    yanalytic[i+1] = y[0]-(vT*vT/g)*log(cosh(g*t[i+1]/vT))+vy[0]*t[i+1]
    if ( y[i+1] < 0.0):
        break
data = {'t[s]': t,
        'Relative error in y': abs((y-yanalytic)/yanalytic),
        'vy[m/s]': vy,
        'x[m]': x,
        'vx[m/s]': vx
}

NewData = pd.DataFrame(data)
err=NewData['Relative error in y']

```

```
display(NewData)
# save to file
NewData.to_csv(outfile, index=False)

#then plot
fig, axs = plt.subplots(4, 1)
axs[0].plot(t, y)
axs[0].set_xlim(0, tfinal)
axs[0].set_ylabel('y')
axs[1].plot(t, vy)
axs[1].set_ylabel('vy[m/s]')
axs[1].set_xlabel('time[s]')
axs[2].plot(t, x)
axs[2].set_xlim(0, tfinal)
axs[2].set_ylabel('x')
axs[3].plot(t, vx)
axs[3].set_ylabel('vx[m/s]')
axs[3].set_xlabel('time[s]')

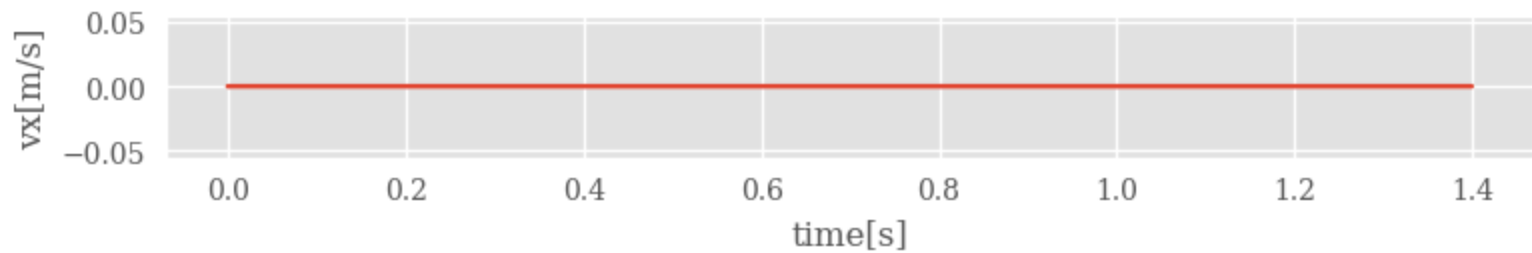
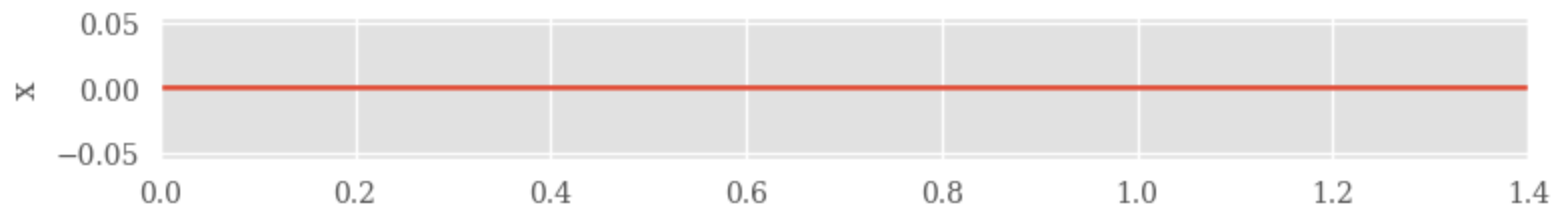
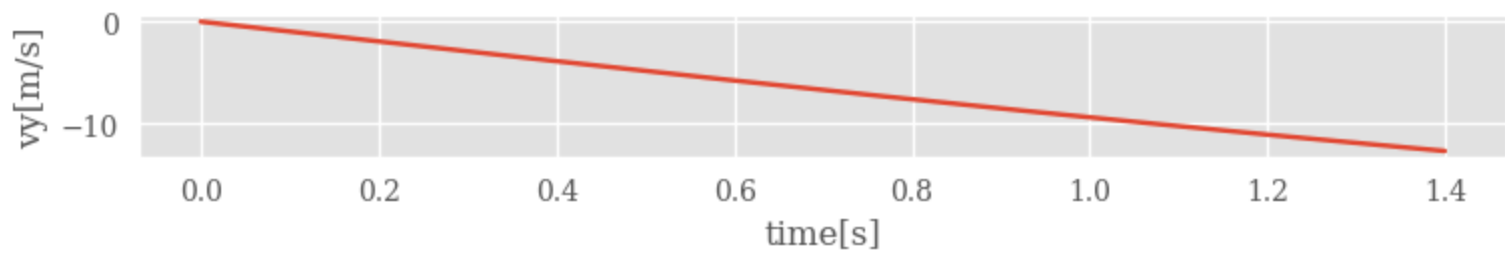
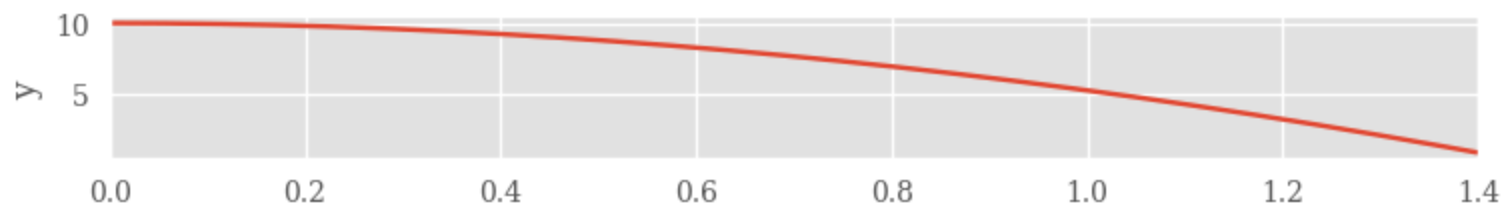
fig.tight_layout()
save_fig("EulerIntegration")
plt.show()

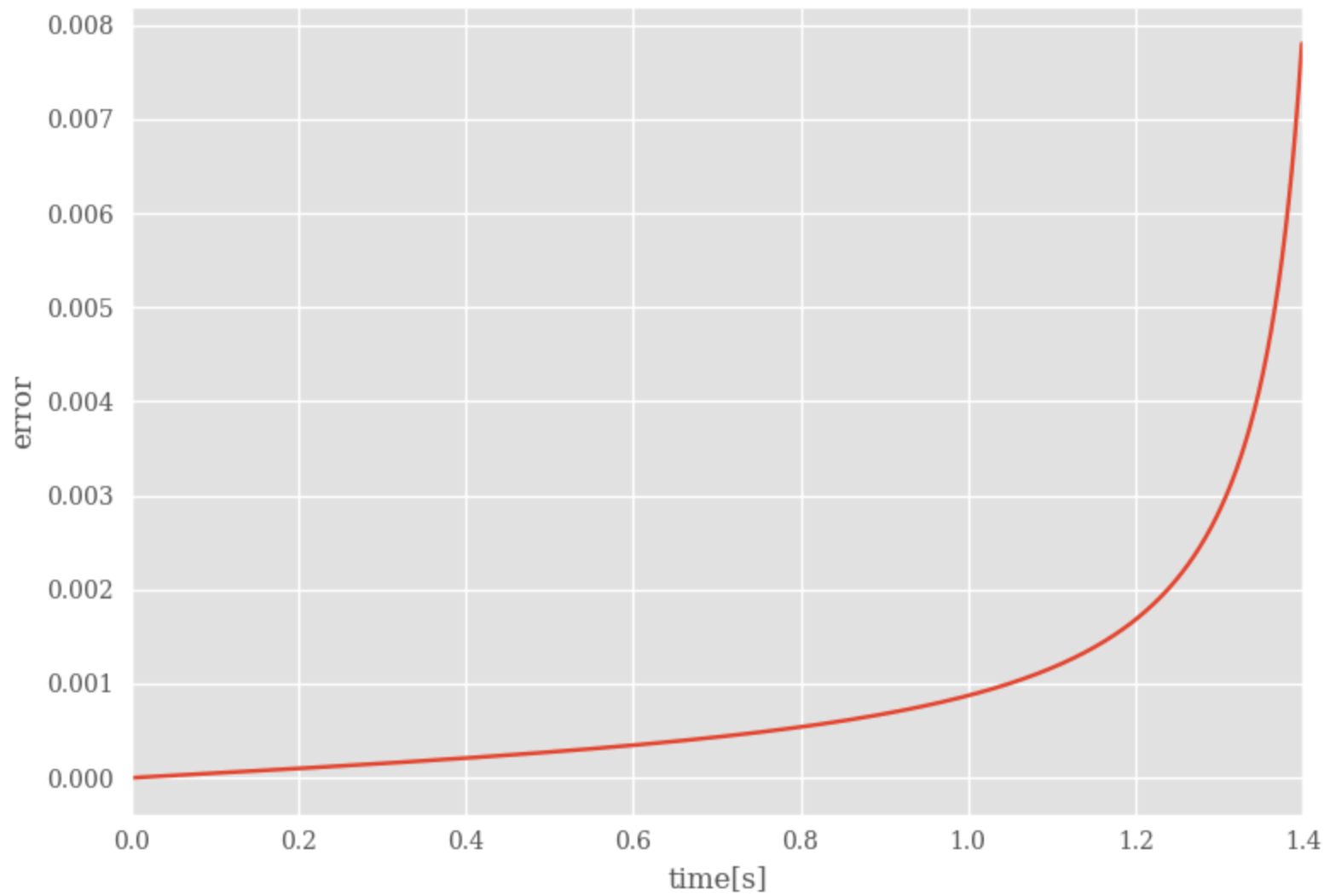
fig2, axs2 = plt.subplots(1,1)
axs2.plot(t, err)
axs2.set_xlim(0, tfinal)
axs2.set_ylabel('error')
axs2.set_xlabel('time[s]')

fig2.tight_layout()
save_fig("ErrorEuler")
plt.show()
```

	t[s]	Relative error in y	vy[m/s]	x[m]	vx[m/s]
0	0.000	0.000000e+00	0.000000	0.0	0.0
1	0.001	4.903277e-07	-0.009807	0.0	0.0
2	0.002	9.806568e-07	-0.019613	0.0	0.0
3	0.003	1.470988e-06	-0.029420	0.0	0.0
4	0.004	1.961324e-06	-0.039226	0.0	0.0
...	...	...	...	...	...
1395	1.395	7.296969e-03	-12.706056	0.0	0.0
1396	1.396	7.417356e-03	-12.713885	0.0	0.0
1397	1.397	7.541724e-03	-12.721712	0.0	0.0
1398	1.398	7.670274e-03	-12.729536	0.0	0.0
1399	1.399	7.803222e-03	-12.737357	0.0	0.0

1400 rows × 5 columns





Our error is not too bad, but it seems like it is growing over time. This is because the Euler method is a first order method, so the error is proportional to Δt . We will need a better method to get more accurate results.